3. TPM 핵심 개념 및 구현 분석

3.1 TPM(Trusted Platform Module) 기본 개념

3.1.1 TPM 개요

TPM(Trusted Platform Module)은 컴퓨터 시스템에 하드웨어 기반 보안 기능을 제공하는 전용 마이크로컨트롤러입니다. Keylime에서 TPM은 다음과 같은 핵심 역할을 수행합니다:

- **플랫폼 무결성 측정**: 부팅 과정과 런타임 시스템 상태를 암호학적으로 측정
- **암호화 키 관리**: 하드웨어 기반 키 생성 및 보호
- **원격 증명(Remote Attestation)**: 시스템 상태를 원격으로 검증할 수 있는 증명 생성

3.1.2 TPM 2.0 지원

Keylime은 TPM 2.0 사양을 기반으로 구현되어 있으며, 하드웨어 TPM과 소프트웨어 TPM 에뮬레이터를 모두 지원합니다:

```
# keylime/tpm/tpm_main.py의 Tpm 클래스

class Tpm:

@staticmethod

def  encrypt_aik_with_ek(uuid: str, ek_tpm: bytes, aik_tmp: bytes) -> Optional[T

# TPM 2.0 기반 AIK 암호화 구현
```

3.2 PCR(Platform Configuration Register) 동작 원리

3.2.1 PCR 기본 구조

PCR은 TPM 내부의 특수한 레지스터로, 시스템의 상태 변화를 암호학적 해시로 기록합니다. Keylime에서 PCR 상태는 `TPMState` 클래스로 관리됩니다:

```
# keylime/agentstates.py

class TPMState:

"""TPMState models the state of the TPM's PCRs"""

pcrs: Dict[int, Optional[bytes]]

hash_alg: Dict[int, Hash]

def  __init__(self) -> None:

self.pcrs = {}

self.hash_alg = {} # PCR별 해시 알고리즘 기록

for pcr_num in  range(0, 24):

self.reset_pcr(pcr_num)
```

3.2.2 PCR 확장(Extend) 연산

PCR 값은 직접 쓰기가 불가능하며, 오직 확장(extend) 연산을 통해서만 업데이트됩니다:

```
# keylime/tpm/tpm_main.py

@staticmethod

def  sim_extend(hashval_1: str, hash_alg: Hash) -> str:

"""Compute expected value H(0|H(data))"""

hdata = hash_alg.hash(hashval_1.encode("utf-8"))

hext = hash_alg.hash(hash_alg.get_start_hash() + hdata)
```

```
return hext.hex()
```

3.2.3 PCR 초기값 설정

각 PCR은 용도에 따라 다른 초기값을 가집니다:

```
# keylime/agentstates.py
@staticmethod
def initial_pcr_value(pcr_num: int, hash_alg: Hash) -> bytes:
"""Return the value for a PCR being initialized"""
if 17 <= pcr_num <= 23:
return hash_alg.get_ff_hash() # 0xFF로 초기화
return hash_alg.get_start_hash() # 0x00으로 초기화
```

3.2.4 PCR 검증 프로세스

Keylime은 `check_pcrs` 메서드를 통해 PCR 값들을 정책과 비교하여 검증합니다:

```
def check_pcrs(
self,
agentAttestState: Optional[AgentAttestState],
tmp_policy: Union[str, Dict[str, Any]],
pcrs_dict: Dict[int, str],
data: str,
ima_measurement_list: Optional[str],
```

```
# ... 기타 매개변수들

) -> Failure:

# PCR 정책 검증 로직

pcr_allowlist = tpm_policy_dict.copy()

# 데이터 PCR 검증

if config.TPM_DATA_PCR in pcr_nums and data is not None:

expectedval = self.sim_extend(data, hash_alg)

if expectedval != pcrs_dict[config.TPM_DATA_PCR]:

# PCR 불일치 오류 처리
```

3.3 EK(Endorsement Key)와 AIK(Attestation Identity Key) 역할

3.3.1 EK(Endorsement Key) 개념

EK는 TPM 제조 시점에 생성되는 고유한 비대칭 키 쌍으로, TPM의 신원을 증명하는 역할을 합니다:

- **고유성**: 각 TPM마다 유일한 EK를 보유
- **불변성**: 제조 후 변경 불가능
- **인증서**: TPM 제조사가 발급한 EK 인증서로 진위 검증

3.3.2 AIK(Attestation Identity Key) 개념

AIK는 증명 작업을 위해 동적으로 생성되는 키로, 익명성을 보장하면서 증명을 수행합니다:

- **익명성**: EK와 달리 추적 불가능한 익명 신원 제공
- **증명 전용**: 오직 증명(attestation) 목적으로만 사용

- **동적 생성**: 필요에 따라 새로운 AIK 생성 가능

3.3.3 EK 검증 프로세스

Keylime은 EK 인증서의 유효성을 검증합니다:

```
# keylime/tpm/tpm_main.py
@staticmethod
def  verify_ek(ekcert: bytes, tpm_cert_store: str) -> bool:
"""Verify that the provided EK certificate is signed by a trusted root
:param ekcert: The Endorsement Key certificate in DER format
:returns: True if the certificate can be verified, false otherwise
"""
return cert_utils.verify_ek(ekcert, tpm_cert_store)
```

3.4 `keylime/tpm/tpm_main.py`의 `Tpm` 클래스 분석

3.4.1 클래스 구조 개요

`Tpm` 클래스는 Keylime의 TPM 관련 핵심 기능을 제공하는 정적 메서드들로 구성되어 있습니다:

```
class Tpm:
# AIK 암호화 관련
@staticmethod
def  encrypt_aik_with_ek(uuid: str, ek_tpm: bytes, aik_tpm: bytes) -> Optional[T
# AIK 검증 관련
```

```
@staticmethod

def verify_aik_with_iak(uuid: str, aik_tpm: bytes, iak_tpm: bytes, iak_attest: b

# Quote 검증 관련

def check_quote(self, agentAttestState: Optional[AgentAttestState], nonce: str,

# PCR 검증 관련

def check_pcrs(self, agentAttestState: Optional[AgentAttestState], tpm_policy: U
```

3.4.2 주요 메서드 분석

Quote 검증 메서드

```
def  check_quote(

self,

agentAttestState: Optional[AgentAttestState],

nonce: str,

data: str,

quote: str,

aikTmpFromRegistrar: str,

# ... 기타 매개변수들

) -> Failure:

# 1. Quote 데이터 검증

pcrs_dict, err = Tpm._tpm2_checkquote(aikTmpFromRegistrar, quote, nonce, str(has

# 2. TPM 클록 정보 검증

if not skip_clock_check and agentAttestState is not None:
```

```
clock_failure, current_clock_info = Tpm.check_quote_timing(...)

# 3. PCR 값 검증

if not skip_pcr_check:

return  self.check_pcrs(...)
```

TPM 클록 정보 검증

```
@staticmethod

def  check_quote_timing(

previous_clockinfo: TPMClockInfo,

quote: str,

compressed: bool

) -> Tuple[Optional[str], Optional[TPMClockInfo]]:

# TPM 클록 정보 추출

clock_info_dict = Tpm._tpm2_clock_info_from_quote(quote, compressed)

tentative_current_clockinfo = TPMClockInfo.from_dict(clock_info_dict)

# 클록 무결성 검증

resetdiff = tentative_current_clockinfo.resetcount - previous_clockinfo.resetcou

restartdiff = tentative_current_clockinfo.restartcount - previous_clockinfo.rest

if resetdiff < 0:

return  "resetCount value decreased on TPM between two consecutive quotes", curr
```

3.5 `encrypt_aik_with_ek()` 함수 동작 과정 상세 설명

3.5.1 함수 시그니처 및 목적

```
@staticmethod
def  encrypt_aik_with_ek(uuid: str, ek_tpm: bytes, aik_tpm: bytes) -> Optional[T
```

이 함수는 TPM의 MakeCredential/ActivateCredential 프로토콜을 구현하여 AIK의 신뢰성을 검증하는 핵심 메커니즘입니다.

3.5.2 단계별 동작 과정

1단계: 입력 검증 및 AIK 이름 추출

```
if ek_tpm is None or aik_tpm is None:
logger.error("Missing parameters for encryptAIK")
return None
```

```
aik_name = tpm2_objects.get_tpm2b_public_name(aik_tpm)
```

2단계: 챌린지 생성

```
# 32바이트 랜덤 챌린지 생성
challenge_str = tpm_util.random_password(32)
challenge = challenge_str.encode()
```

```
logger.info("Encrypting AIK with EK for UUID %s", uuid)


# Base64로 인코딩하여 키로 사용
key = base64.b64encode(challenge).decode("utf-8")
```

3단계: Credential Blob 생성

```
credentialblob = tpm_util.makecredential(ek_tpm, challenge, bytes.fromhex(aik_na
keyblob = base64.b64encode(credentialblob)
```

3.5.3 `makecredential` 함수 상세 분석

`tpm_util.makecredential` 함수는 TPM 2.0의 MakeCredential 명령을 소프트웨어로 구현합니다:

```
def makecredential(ek_tpm: bytes, challenge: bytes, aik_name: bytes) -> bytes:
"""TPM_MakeCredential implementation"""
# 1. EK 공개키 및 해시 알고리즘 추출
public_key, hash_alg = tpm2_objects.pubkey_parms_from_tpm2b_public(ek_tpm)
hashfunc = tpm2_objects.HASH_FUNCS.get(hash_alg)
# 2. 키 타입별 암호화 처리
if isinstance(public_key, RSAPublicKey):
# RSA 키의 경우 OAEP 패딩 사용
random = os.urandom(hashfunc.digest_size)
secret = public_key.encrypt(
```

```
random,
padding.OAEP(
mgf=padding.MGF1(algorithm=hashfunc),
algorithm=hashfunc,
label=label_to_bytes("IDENTITY")
),
)
elif  isinstance(public_key, EllipticCurvePublicKey):
# ECC 키의 경우 ECDH 사용
random, secret = crypt_secret_encrypt_ecc(public_key, hashfunc)
# 3. Credential Blob 생성
credentialblob = secret_to_credential(challenge, aik_name, random, ek_tpm, hashf
# 4. tpm2-tools 호환 형식으로 패키징
hdr = struct.pack(">II", 0xBADCC0DE, 1)
tail = struct.pack(f">H{len(secret)}s", len(secret), secret)
return hdr + credentialblob + tail
```

3.5.4 보안 메커니즘

암호화 계층 구조

1. **외부 암호화**: EK 공개키로 대칭키 암호화
2. **내부 암호화**: 대칭키로 실제 챌린지 암호화
3. **무결성 보호**: HMAC으로 데이터 무결성 보장

키 유도 과정

```
def  secret_to_credential(
secret: bytes, name: bytes, seed: bytes, ek_tpm: bytes, hashfunc: hashes.HashAlg
) -> bytes:
# 1. 민감한 데이터를 TPM2B 형식으로 패키징
sensitive_data_2b = struct.pack(f">H{len(secret)}s", len(secret), secret)
# 2. 외부 래핑 수행 (암호화 + MAC)
integrity, sensitive_data_enc = produce_outer_wrap(ek_tpm, name, hashfunc, seed,
# 3. TPMS_ID_OBJECT 구조체 생성
tpms_id_object = struct.pack(
f">H{len(integrity)}s{len(sensitive_data_enc)}s",
len(integrity), integrity, sensitive_data_enc,
)
return struct.pack(f">H{len(tpms_id_object)}s", len(tpms_id_object), tpms_id_obj
```

3.5.5 반환값 및 활용

함수는 다음 두 값을 반환합니다:

- `keyblob` : Base64로 인코딩된 credential blob (TPM에서 ActivateCredential로 복호화)
- `key` : Base64로 인코딩된 원본 챌린지 (검증용)

이 메커니즘을 통해 Keylime은 다음을 보장합니다:

1. **AIK 소유권 증명**: 해당 TPM만이 credential을 복호화할 수 있음
2. **EK 연결성**: AIK가 특정 EK와 연결된 TPM에서 생성되었음을 증명

3. **재생 공격 방지**: 매번 새로운 챌린지 사용

이러한 암호학적 프로토콜을 통해 Keylime은 원격 시스템의 TPM 기반 신뢰성을 안전하게 검증할 수 있습니다.